2 - Test automation and continuos testing

Software quality assurance

Si definisce **software quality assurance** l'insieme delle attività sistematiche che forniscono prova dell'idoneità all'uso concepito dell'intero prodotto software. In altre parole, è un processo nel quale si definiscono dei punti di controllo per misurare degli obiettivi (misurabili) da raggiungere.

Dalla definizione originale:

> "SQA is the systematic activities providing evidence of the fitness for use of the total software product."

- systematic activities -> SQA è un processo
- providing evidence -> sono necessari punti di controllo
- fitness -> si pongono degli obiettivi misurabili da raggiungere.

Tale processo, composto da piani e attività, è eseguito durante la fase di sviluppo del software per il monitoraggio e controllo dello sviluppo. L'obiettivo è assicurare, in un certo intervallo di confidenza, che il prodotto software:

- soddisfi le sue specifiche
- sia utilizzabile nel contesto richiesto
- sia sviluppato secondo le modalità stabilite

La creazione di un sistema software di qualità implica l'adozione in un intero processo che abbia l'obiettivo di assicurare la confidenza tra quanto si sta realizzando e le specifiche ad ogni passo. Ad esempio, si può prendere come specifica degli artefatti provenienti dall'analisi per andare incontro ad aspetti di validazione, oppure specifiche del modello di progettazione/implementazione per andare incontro ad aspetti di verifica.

La SQA è definita da tre componenti:

- **software testing** -> è solo uno degli aspetti del SQA, ma da solo non è sufficiente
- **quality control** -> pianificazione delle attività di controllo
- **software configuration management** -> pianificazione per la gestione degli artefatti software che si stanno elaborando.
  In particolare, gli strumenti di software testing non devono essere visti come standalone, ma sono integrati a tutte le fasi di sviluppo, dall'analisi al deployment.

Esempio

Si consideri di conoscere solo la firma del metodo `myBuggyFact`. Non sapendone l'implementazione, eseguire il testing su tutti i possibili input richiederebbe l'implementazione di 2^{32} casi di test. Invece, avendo a disposizione di un

Automated testing

Le attività di software testing sono tutte automatizzabili:

- generazione
- selezione
- prioritizzazione
- esecuzione
- report
- valutazione della bontà/validità di un dato insieme di casi d i test (trovare un oracolo per il sistema, applicare inferenza, utilizzare degli storici).

Queste attività automatizzate sono contrapposte a quelle fatte a mano dal team di QA e/o dall'utente finale (**manual testing**).

L'obiettivi dell'AT sono:

- migliorare l'efficienza dei processi di software testing
- migliorare l'efficacia dei processi di software testing

I benefici principali sono:

- riduzione dei costi
- riduzione della durata delle fasi di test
- aumento della reperibilità degli esperimenti considerati
- aumento dell'attendibilità statistica dei responsi delle fasi di QA
- miglioramento della documentazione delle failure.

Quality control

Il **quality control** è la definizione di un piano per il monitoraggio del processo di sviluppo e rilascio e il controllo del soddisfacimento dei requisiti. Gli strumenti utilizzati sono:

- il processo di Quality Assurance (QA)
- metodi di monitoraggio
- controlli di QA
  È necessaria che ci sia una definizione esplicita del processo di Quality Assurance accanto al processo di progettazione e sviluppo. Ad esempio, i requisiti dovranno essere scritti in un certo modo per poterli utilizzare nel piano di Quality Assurance per derivare i casi di test. Nella fase di analisi, devono essere prodotti un certo numero di documenti fatti in un certo modo da essere utilizzati come specifica nella fase di test.

Software configuration management

Il **software configuration management** è composto a sua volta da una serie di blocchi che indicano di sviluppare il software in un certo modo:

- **component identification** -> il software deve essere scritto in maniera più modulare possibile
- **version control** -> serve un meccanismo automatico di controllo delle versioni; senza questo, non è possibile fare SQA e testing in modo appropriato, poiché l'errore potrebbe non essere nel programma, ma nel modo i cui sono state messe insieme (manualmente) le versioni
- **configuration building** -> non è possibile creare un prodotto di qualità se sviluppato solo su una macchina, poiché il sistema stesso dipende dall'ambiente in cui è stato sviluppato; è necessaria una gestione delle configurazioni esplicita nelle fasi di build che astraggono il più possibile la configurazione del sistema dalla macchina.
- **change control**

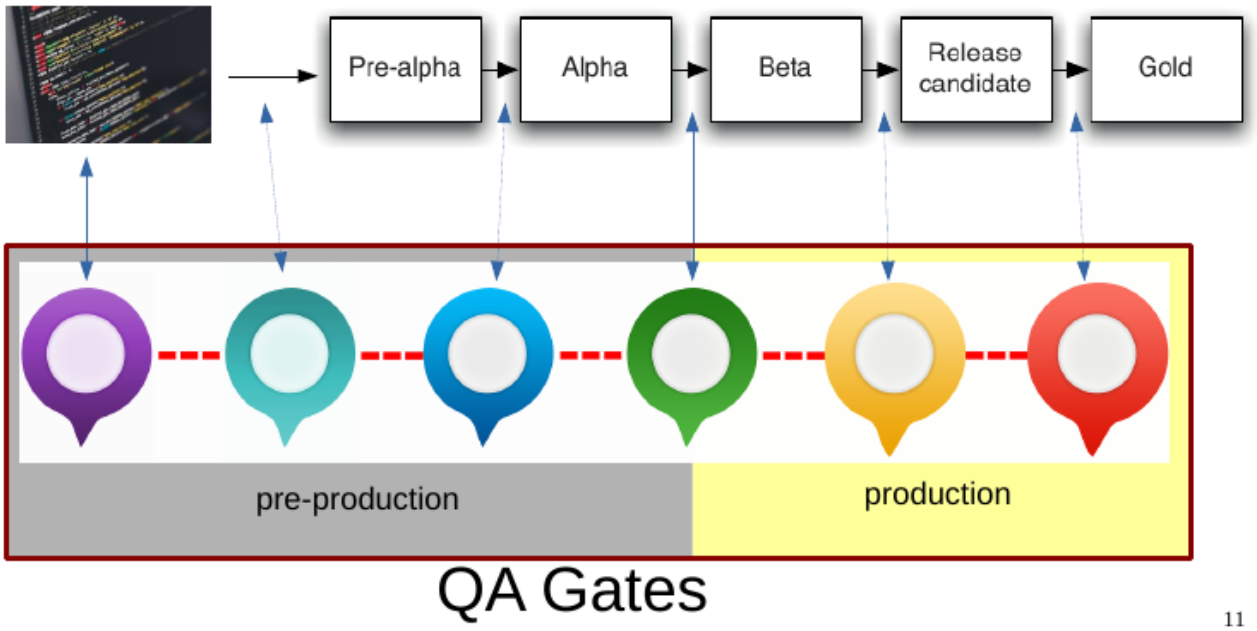
SQA e ciclo di rilascio - approccio tradizionale

È possibile immaginare il processo di quality assurance come una serie di tappe intervallate da dei **QA gates**, ossia obiettivi che indicano come deve essere fatto il prodotto fino a quel punto. Ogni QA gate definisce un momento di controllo con degli obiettivi misurabili, che possono riguardare la struttura del processo, gli outcome del processo o l'esecuzione del sistema software.

I QA gate scandiscono delle macroversioni del sistema. Le principali sono:

- **pre-aplha** -> sono stati raggiunti un certo grado di maturità, un certo grado di code smells, un certo numero di test, un certo grado di confidenza dei profili operazionali utilizzati per configurare il sistema ecc.
- **alpha** -> versione quasi finale del sistema testata dagli sviluppatori e dai provider in ambiente controllato
- **beta** -> versione rilasciata sul mercato e provata da un insieme ristretto di utilizzatori con i profili e le condizioni di utilizzo degli utilizzatori, ma non ancora la versione ufficiale
- **release candidate** -> versione che è quasi in uscita

- **gold** -> versione ufficiale



Nella pratica, succede spesso che il sistema sembri funzionare, ma appena rilasciato in ambiente operativo risulta non più funzionante. Ciò è probabilmente dovuto al fatto che il sistema è stato provato in un sottoinsieme di ambienti e scenari possibili. Il problema principale di questi tipi di scenari è vedere i QA gate in maniera discreta: la verifica degli obiettivi nel QA gate può andare a buon fine o meno. Tale discretizzazione, seppur utile, tende a creare questo tipo di problemi.

Nel tempo l'approccio è stato cambiato per percepire i controlli come un processo continuo e non più discreto. In particolare, ogni cambiamento deve essere percepito come una potenziale release del sistema. Ciò significa che tutte le volte che si aggiunge una funzionalità, il sistema software deve essere visto come un sistema complesso in cui il sistema di QA applica tutti i controlli previsti dall'inizio alla fine. Questo ha un maggiore costo in termini di soldi e tempo. Quando si sviluppa, occorre farlo in maniera intelligente e trasparente, in modo che ogni volta entrino in gioco i processo di QA in maniera automatica.

Automated testing

Per **automated testing** si intendono tutte le attività svolte per mezzo di infrastrutture o framework per il software testing che servono ad automatizzare:

- l'esecuzione e il report delle attività di test
- la selezione e la prioritizzazione dei test da eseguire
- la valutazione della bontà e della validità di un insieme di test case

L'automated testing è una parte di QA che porta automazione su ognuno di questi aspetti, cioè cercando di contribuire al claim *“every contribution leads to a potential release”*, quindi di cerca di contribuire in modo automatico per controllare in modo automatico che il software rispetti le specifiche. È possibile fare automated testing in diversi modi:

- strumenti che automatizzano i report e le attività di test -> ad ogni commit, vengono eseguiti in automatico tutti i test e viene generato un report
- altri strumenti generano in automatico test case da eseguire, oppure li selezionano o il prioritizzano (questo poiché lanciare i test può costare anche tanto)
- strumenti per valutare la bontà e la validità dei test case, che indicano quanto sono utili i test creati

L'automated testing si contrappone al testing svolto da umani (**manual testing**).

I principali obiettivi di AT sono migliorare l'efficienza e l'efficacia dei processi di QA. l'AT porta anche i seguenti benefici:

- riduzione dei costi e della durata delle fasi di test
- aumento della reperibilità degli esperimenti considerati
- aumento dell'attendibilità statistica dei responsi delle fasi di QA
- migliorare la documentazione delle failure.

L'automated testing si pone nel discorso *“every contribution leads to a potential release”* tramite **continuous testing**.

Continuous testing

Il **continuous testing** si occupa dell'integrazione dei processi di QA (in particolare dei processi di software testing) con:

- l'ambiente di sviluppo
- gli strumenti di build
- i sistemi di versioning

In particolare, si parla di strategia di testing in cui i test vengono eseguiti automaticamente sull'attuale versione del sistema, in particolare:

- su configurazioni locali allo sviluppo -> per far sì che il sistema funzioni localmente, al fine di continuare lo sviluppo
- su ambienti simili agli ambienti di produzione finali -> per garantirne il corretto funzionamento al rilascio

L'esecuzione dei test avviene in maniera trasparente. Infatti, gli sviluppatori non devono preoccuparsi di avviare esplicitamente le fasi di test, poiché questi vengono eseguiti dopo ogni comprovata evoluzione della code base.

I risultati de test sono notificati in modalità asincrona rispetto al termine dell'esecuzione, in modo da non interrompere lo sviluppo. Inoltre, è necessario prevedere delle policy che regolino quali test effettivamente eseguire: selezione, prioritizzazione, orchestrazione dei casi di test.

Integrazione di CI e CT

Si consideri un ciclo di svluppo definito dal seguente schema di build:

Lo sviluppatore sviluppa una nuova parte di codice e fa un commi. Il VCS riconosce il cambiamento di versione del sistema e genera un branch per le nuove funzionalità sviluppate dal dipendente e sulle quali eseguire i test. Allora, il VCS avvia una serie di attività di QA, in particolare di testing su un build server (server in grado di tirare su una serie di macchine virtuali con diversi SO). L'ambiente di testing è disaccoppiato da quello di sviluppo e punta ad essere il più simile possibile all'ambiente di deployment finale. Il testing genera un report che viene poi inviato allo sviluppatore, in modo che abbia conoscenza di eventuali problemi relativi alle funzionalità da lui sviluppate. Tutto ciò deve avvenire in maniera trasparente allo sviluppatore.

In particolare, il build server include:

- configurazione di un ambiente tipo
- build del sistema
- esecuzione dei test case
  Il test engineer deve strutturare il processo di CT che consenta allo sviluppatore in maniera indipendente al software testing.

Un simile approccio definisce quindi una tecnica di **CI + CT**:

- **CI** -> integrazione di pezzi di sistema diversi (si verifica se le funzionalità sviluppate sono ammissibili o meno)
- **CT** -> esecuzione continua di test sulla codebase (si fa partire - potenzialmente - l'intero processo di testing su tutta la codebase)

Queste attività sono eseguibili sono in un contesto di automated testing, ossia con avendo degli strumenti di VCS, generazione di test e report ecc.

Framework

Version control : Git

Git è il VCS più noto (ref). È open-source e distribuito.

L'importante è fare un branch del master su cui lavorare, poi fare il merge delle funzionalità sul master.

Diversi scenari:

- repository condiviso -> i vari sviluppatori eseguito i commit su un repository condiviso
- c'è un repository "benedetto" (o ufficiale della versione corrente del software come sorgente, ma non quello di release) che ha la responsabilità di mantenere la copia effettiva in sviluppo. Gli sviluppatori hanno delle copie di questo repository, su cui essi lavorano. È anche presente un gestore dell'integrazione (o integratore di sistema) che prende le parti sviluppate da ogni sviluppatore ed eventualmente ne fa il merge sul repository benedetto.
- in alcuni casi, l'integratore svolge la funzione di dittatore benevolo, cioè colui che controlla effettivamente lo sviluppo del sistema, ergo una modifica viene integrata solo se il dittatore decide che può essere integrata. I processi di QA possono prevedere nell'integratore dei test, ma esso può anche prevederne di suoi.
  Per eseguire i test, l'integratore crea un ulteriore fork della versione corrente su cui fa il merge del contributo. A quel punto, esegue i test ed eventualmente fa il merge nel ramo principale.

L'introduzione di un integratore è necessaria per avere un controllo (statico o dinamico) sull'integrazione e il testing del software.

Configuration building: Maven

Maven (ref) è uno strumento di build sviluppato da Apache Software Foundation. È uno strumento che si occupa di fare il build, compilare, aggregare ed eventualmente fare anche il deployment di sistemi software. Nasce principalmente per Java, ma è agnostico al sistema di sviluppo.

Maven implementa il paradigma del component-based development/design, ossia gestisce in maniera indipendente e transitiva l'import di librerie. È in grado infatti di recuperare in automatico le librerie di secondo livello necessarie a quelle di primo livello dichiarate dallo sviluppatore tramite degli appositi registry, aggiungendole anche al classpath in fase di compilazione.

Un progetto Maven è organizzato in un particolare schema di directory nel file system. Dal punto di vista logico sono delle directory collegate in un certo modo. La struttura delle directory è definita un **archetipo** (*archetype*). L'archetipo Java prevede la strutturazione in directory per i sorgenti e altre per i test.

Un progetto Maven può essere diviso in **moduli**, ossia separato in più parti che collaborano tra di loro, al fine di organizzare meglio la struttura del codice, la responsabilità di chi ci mette mano e i permessi e i processi di QA. I moduli possono essere dichiarati internamente o esternamente alla directory principale. I moduli sono esplicitati nella dichiarazione del progetto.

Tutte le informazioni di un progetto o modulo Maven sono raccolte in uno specifico descrittore, chiamato `pom.xml`. Questo file serve a battezzare il progetto/modulo, le dipendenze del progetto/modulo, quello che deve essere fatto nelle varie fasi di build, test, deploy del progetto/modulo. Tipicamente, il file introduce:

- artifactId
- groupId
- nome
- versione
- sviluppatori
- dipendenze
- direttive di build
  Spesso include anche direttive di configurazione, di test e di deploy.

L'output di un modulo Maven viene inserito in una directory chiamata `target`. In particolare, qui vengono inseriti tutti i risultati di una qualsiasi fase di Maven. Su un VCS non deve essere presente la directory `target`: se, ad esempio, i risultati in target vengono generati in parallelo da due sviluppatori su due macchine Linux e Windows, allora nell'eseguire i push ci saranno dei conflitti su quella cartella.

Nota: il comando `make clean` ripulisce la cartella `target` di tutti i risultati prodotti da Maven nelle sue fasi.

Maven non lavora solo localmente all'ambiente di sviluppo. Esso è un sistema distribuito di cui si scarica un client. Esiste un sistema distribuito globale di registri, chiamato **Maven Central**, che mantiene copie delle librerie Maven. Quando è necessaria una libreria non presente localmente:

- viene interrogata un'istanza di Maven Central (o un'altra specificata dall'utente)
- viene recuperata la copia richiesta della libreria
- tale copia viene salvata nella cache locale di Maven (directory `.m2`)
- viene inserito il path della libreria nel classpath per la compilazione

Maven è pensato per gestire l'intero processo di build. Questo è strutturato in fasi, concettualmente organizzate in sequenza. Tipicamente, errori nell'esecuzione di una fase comportano il fallimento dell'intero processo di build. Il risultato di una qualsiasi fase viene posto di default nella directory `target` del progetto. Le fasi di default di Maven sono:

- `validate` -> verifica che il progetto sia corretto e che tutte le informazioni necessarie siano disponibili
- `compile` -> compila il codice sorgente del progetto
- `test` -> testa il codice compilato utilizzando un framework di unit testing adatto
- `package` -> prende il codice compilato e lo impacchetta in un formato distribuibile, ad esempio JAR
- `integration-test` -> processa ed esegue il deployment il package se necessario in un ambiente in cui può essere eseguito l'integration testing
- `verify` -> esegue e controlla per verificare che il package sia valido e vada incontro ai criteri di qualità
- `install` -> installa il package nella repository locale, per utilizzarlo come dependency in altri progetti locali
- `deploy` -> eseguito in ambiente di integrazione o rilascio, copia il package finale alla repository remota
- `clean` -> pulisce gli artifact creati dall'ultima build dalla directory `target`
- `site` -> genera la site documentation per il progetto
  È anche possibile dichiarare ulteriori fasi.

Tutorial in 5 minutes. Reference.

Self-testing: JUnit

JUnit è un framework open source che supporta l'implementazione di test program in ambienti Java (ref). L'introduzione di JUnit ha contribuito:

- a rendere i test (specialmente di unità) una parte centrale nella fase di programmazione di un software
- allo sviluppo di tecniche di programmazione basate su testing (come TDD, Test Driven Development)

Si usa dire che JUnit è un DSL (Domain-Specific Language) che raffina Java inserendo concetti relativi al software testing, quali standard e dialetti specifici.

Tra i requisiti principali di JUnit figurano:

- supporto alla definizione di test case utili
- supporto alla definizione di test case ripetibili nel tempo
- riduzione del costo della scrittura dei test case attraverso pratiche orientate al riuso del codice

Tuttavia, JUnit non indica quale strategia di test usare, quali test siano più significativi o quali valori di input adottare o selezionare.

Disponibili diverse versioni:

- `4.x` -> la versione più diffusa
  - i test sono dichiarati per mezzo di annotazioni
  - sono previsti costrutti per il supporto del setup e il teardown del contesto di esecuzione
  - include strumenti per la gestione esplicita di come un insieme di test case debba essere eseguito (ossia, i **runners**)
- `5.x` -> la versione più recente
  - modularità dell'architettura aumentata
  - consente la composizione esplicita di più runners
  - differenzia due principali tipologie di use-case:
    - scrittura di programmi di test tramite l'invocazione di un'API
    - estensione della SPI (Service Provider Interface) per supportare diverse strategie per la scoperta/scelta ed esecuzione di test case

JUnit 4

In JUnit 4, un test case è un metodo pubblico annotato con `@Test`.

Le annotazioni `@Before` e `@After` indicano quali azioni eseguire prima e dopo ogni test:

- l'esecuzione di metodi annotati in questo modo non dipende dai risultati dei test
- non è specificato un ordine di esecuzione tra più metodi annotati con `@Before`, oppure tra metodi annotati con `@After`
  Solitamente, i metodi etichettati con `@Before` pongono delle precondizioni per i test, mentre quelli annotati con `@After` eseguono delle operazioni di cleanup a seguito dei test. Si può forzare JUnit per fare in modo che i metodi `@Before` e `@After` vengano eseguiti in maniera random per assicurarsi che non siano presenti bias dovuti alla macchina locale. Il fatto che l'ordine sia non deterministico è dovuto a motivi di prestazioni, essendo il tutto gestito tramite reflection e dipendendo dal modo in cui vengono popolate le strutture dati.

Le annotazioni `@BeforeClass` e `@AfterClass` indicano azioni da eseguire una sola volta prima o dopo l'esecuzione di tutti i metodi annotati con `@Test` all'interno della classe. Come per `@Before` e `@After`, non è specificato un ordine di esecuzione. Questi metodi sono statici.

Se si utilizza JUnit 4 insieme a Maven, i nomi delle classi di test devono rispecchiare i seguenti pattern:

- `Test*`

- `*Test`
- `*Tests`
- `*TestCase`
  Infatti, per gli archetipi Java, Maven riconosce le classi di test in base al pattern del nome della classe.

Continuous integration

Piattaforme per la continuous integrazione e la continuous delivery sono:

- TRAVIS-CI (ref)
- GitHub Actions (ref)
- GitLab CI-CD (ref)
- Circle-CI (ref)

I framework di CI sono offerti come servizi remoti e prevedono policy e costi di utilizzo:

- spesso non ci sono costi per progetti open-source
- se presenti, le limitazioni di utilizzo imposte su account freeware sono spesso sovrabbondanti per gli scopi del corso e non è richiesta alcuna sottoscrizione a pagamento

In generale, ogni framework di CI ha le sue specifiche procedure di configurazione rispetto al VCS utilizzo.